

Lab 3: Gesture Recognition on the Arduino Nano

last update: 6/22/26

General Information

Lab Mode: Remote. Work on the lab remotely using your own computer and lab dev kit.

Lab Due Date: Posted in Canvas. Submit all the required deliverables by the due date.

Lab Report Submission: Submit via Canvas. No email submissions are accepted.

Lab Contents: Additional requirements applied for EECE.5862. Details are below.

Single-track lab. Everyone runs the same IMU-based gesture-recognition pipeline on the Nano 33 BLE Sense Rev2's on-board **BMI270** 6-axis IMU.

Lab Description

Requirements for both EECE.4862 and EECE.5862 students

Lab 2 deployed a windowed classifier — one prediction per gesture. Lab 3 is the capstone: a model that classifies **continuously** on a sliding window of live IMU data, smooths the noisy per-window predictions through a **test-and-trace** post-processing filter, and acts on stable detected events. You also do an **optimization study** (Lecture 9) and report the **real-time performance metrics** (Lecture 10) that decide whether your model is shippable. The lab has three parts: **Part A — Sense**, **Part B — Infer + Act**, and an **Optimization Study** baked into Part B.3.

Complete the following requirements:

- (1) **Part A.1 — Read the IMU and stream it over Serial.** Starting from `starter/g1-imu-capture/`, initialize the BMI270 via `Arduino_BMI270_BMM150` and, at a configurable **sampling rate** (start with **100 Hz** — the BMI270's native accelerometer rate), read the 3-axis accelerometer **and** 3-axis gyroscope, then print them to Serial as one **CSV line** per sample at **115200 baud**: `millis,ax,ay,az,gx,gy,gz`. Use **115200 baud** (not 9600) — at 100 Hz × 6 channels × CSV overhead the 9600 stream backs up and the Edge Impulse data forwarder drops samples. **Deliverable:** the sketch (`g1-imu-capture.ino`), a Serial Monitor screenshot showing ≥ 20 lines of streaming data while you wave the board, and the **measured sample rate** (count lines per second from a one-second capture — it should be close to 100 but won't be exact).
- (2) **Part A.2 — From raw IMU samples to model features.** In your report, explain (½–1 page, with a diagram) how the raw stream becomes the model's **input tensor**. At 100 Hz over a 2-second gesture window, raw IMU is 1,200 scalars — too many for a small dense net to learn from and too noisy to generalize across people. Edge Impulse's **Spectral Analysis DSP block** does the feature reduction for you. Cover: (a) what the block computes — for each of the 6 channels it extracts a small number of time-domain statistics (RMS, peak) and frequency-domain features (FFT magnitudes at binned frequencies, plus spectral power summaries); roughly how many features per channel does the default produce, and therefore how many total features per window? (b) why a frequency-domain representation is useful here — tie it explicitly to a concept from **Lecture 6** (sampling, Nyquist, the affine sensor model) or **Lecture 7** (translation-invariance in time, dimensionality reduction). (c) a small pseudocode/figure showing how 1,200 raw scalars become the ~50–60 features the model actually receives, and identify which step you would have to write yourself if you weren't using Edge Impulse.

- (3) **Part B.1 — Train the model in Edge Impulse.** Build a **Spectral Analysis + dense (or small 1-D CNN) impulse** in **Edge Impulse** to recognize **three gestures** of your choice **plus an idle background class** (four classes total). Suggested set: `punch`, `flex`, `circle`. The idle class — board sitting still or moving slowly while the user walks around — prevents every small wobble from triggering a spurious gesture at deployment time. Record **$\geq \sim 30$ examples per class** (≥ 120 samples total), each ~ 2 seconds long, using either the `edge-impulse-data-forwarder` **CLI** against your A.1 sketch or the **Edge Impulse phone app**. Design the impulse: time-series input **window 2000 ms / stride 500 ms** at your A.1 sample rate (100 Hz typical); **Spectral Analysis** processing block (defaults); **Classification** learning block (default small dense or 1-D CNN); set the FFT length in the Spectral Analysis block to **at least 32** (the nRF52840's hardware FFT requires powers of $2 \geq 32$ — 16 will fail at runtime). Train ~ 100 epochs. Build a **Quantized (int8)** Arduino library from *Deployment* and record the on-device flash / RAM / latency **estimates** the Studio reports for the Nano. **Deliverable:** screenshots of the **impulse design** page, the **Feature explorer**, the **Model** page (validation accuracy + confusion matrix), and per-class **precision / recall / F1** from *Model testing*. State the four classes you used.
- (4) **Part B.2 — Deploy with continuous classification + test-and-trace + act.** Install the EI Arduino library .zip you built in B.1 via **Sketch → Include Library → Add .ZIP Library...**, then start from `starter/g2-gesture-inference/`. The scaffold sets up the IMU sample loop and the EI continuous-classifier call; complete the `// TODO(student)` blocks so that (a) inference runs continuously, once per window stride (500 ms by default); (b) the **test-and-trace** post-processing filter from Cookbook Ch 9 maintains a small history (e.g. last **3 classifications**) and emits a **stable detected gesture** only when **at least 2 of the last 3** windows agreed on the same non-idle class **above your confidence threshold** (start at 0.7); (c) the act stage blinks the on-board **RGB LED** in a per-gesture color (e.g. `punch` → red, `flex` → green, `circle` → blue, `idle` → off), and **throttles re-fire** so a held gesture doesn't re-blink every 500 ms — require a return to `idle` (or 1 second of silence) before re-arming. The handout's **"four filter knobs"** table in `README.md §B.2(b)` documents `kHistoryN`, `kAgreeMin`, `kConfidenceMin`, and `kReArmIdleMs` and which one to tune for which symptom. The Edge Impulse runtime drives the IMU sample loop and the classifier from a single Mbed OS thread for you — you do not need to spawn threads explicitly. **Deliverable:** the completed sketch (`g2-gesture-inference.ino`), and a short **video or stitched screenshots** of the board responding to live gestures — each gesture triggering its matching LED color, `idle` producing no output.
- (5) **Part B.3 — Measure baseline and run an optimization study.** Report a small table in your lab report with your **baseline** numbers on the Nano: per-class accuracy from EI *Model testing*; end-to-end latency from EI's `result.timing` (DSP + classification, **min/mean/max** over ≥ 100 inferences) — note that `result.timing` measures inference cost only; your user-perceived detection latency adds `window_stride × kAgreeMin` of filter latency; classifications/sec (count `Serial.print` lines per second of stable measurement); flash and RAM usage from the Arduino IDE compile output. Then apply **one optimization** from Lecture 9 and re-measure. Pick **one**: (i) **Quantization variant**. Re-export EI's deployment as **unoptimized (float32)** and re-measure; expected direction: `int8` smaller and faster, small accuracy hit. (ii) **Window size**. Halve the window (2000 ms → 1000 ms) in the impulse design, retrain, re-deploy, re-measure; expected direction: smaller window → faster inference, smaller arena, possibly worse accuracy on slower gestures. (iii) **EON Tuner**. Run the **EON Tuner** in Edge Impulse to search for a smaller/faster architecture at similar accuracy, deploy its winner, re-measure. Then write **one short paragraph** (3–5 sentences) tying the result to a specific L9 / L10 idea — name the technique (operator fusion, `int8` PTQ, NAS, window-as-bandwidth-knob) by name. **Deliverable:** the **baseline + after-optimization measurement table** (4 columns: accuracy, latency mean, flash, RAM — two rows) and the analysis paragraph.

Note: (1) Begin Part A with a brief *model* of the data → feature transform; Part B with a block diagram of the *sense* → *infer* → *act* loop including the test-and-trace filter. (2) Use the **Arduino C/C++** programming model (`setup()` / `loop()`, libraries via the IDE). Skeleton starter sketches (file structure + `// TODO(student)` hints) for the sketches above are in the lab repository at <https://github.com/ACANETS/eece-4862-5862-labs> under `lab-3/starter/`. The skeletons compile/upload as-is so you can confirm your toolchain (g3 will need the EI library installed first), but credit comes from your implemented logic: a starter submitted with its TODOs

unfilled earns no credit for that part. **(3)** Your **Rev2** board uses the **BMI270** IMU (Arduino_BMI270_BMM150); wherever the *TinyML Cookbook* shows the Rev1 Arduino_LSM9DS1, the same IMU object and method signatures work after the include swap.

Additional requirements for EECE.5862 students

5862 students must complete **one** of the two options below (4862 students may attempt one for up to +2 bonus credit, capped at the lab's 20-point maximum).

- (6) **Option C.1 — BLE telemetry (the AIoT pattern).** Turn your gesture detector into a small **AIoT end-point** that publishes a one-byte gesture event over **BLE** every time the test-and-trace filter emits a stable detection. The raw IMU stream **never leaves the device** — only the inference *result* does (per Lecture 11 and HW 4 Q8). Start from `starter/g3-ble-telemetry/`, which extends `g2` with a Gesture BLE service (ArduinoBLE library), a `GestureEvent` BLERead/BLENotify characteristic, and a connection-state LED indicator (slow blue pulse while advertising, solid blue while connected). Receive the stream with the provided **Web Bluetooth receiver page** in `tools/web-ble-receiver.html` — open in Chrome or Edge (desktop or Android; iOS Safari does not support Web Bluetooth — iPhone-only students should use **Bluefruit LE Connect** as the open-source fallback). Setup: `cd tools && python3 -m http.server 8080`, then open `http://localhost:8080/web-ble-receiver.html`, click **Connect**, and confirm that performing each gesture emits the matching byte exactly once per detected event. Two paragraphs in your report: (a) **what the radio actually transmits** — quantify the data-rate savings vs. streaming raw IMU (6-channel float32 at 100 Hz over BLE is ~19 Kbit/s; your telemetry is a handful of bytes per *gesture event* — orders of magnitude less); tie to the L11 AIoT power-budget argument (radio is the most expensive component — HW 4 Q7); (b) **one specific privacy risk** the system still has even though raw IMU never leaves the device (the gesture stream is itself behavioral data; tie to L12 *Responsible AI at the Edge* and HW 4 Q12), and describe one mitigation you would add for a shippable product. **Deliverable:** the completed `g3-ble-telemetry.ino`, a **screenshot of the receiver page** showing notifications arriving as you gesture, and the two analysis paragraphs.
- (7) **Option C.2 — Throughput + energy budget at three sample rates.** Characterize the **throughput-vs-accuracy** tradeoff for your gesture detector at **three IMU sampling rates** — for example **50 Hz**, **100 Hz** (the A.1 baseline), and **200 Hz** (the BMI270 supports these native ODRs). This option uses **only on-board measurements plus datasheet lookups** — no extra hardware is required. For each rate, re-deploy and measure: test-set accuracy (retrain in EI with samples at that rate, or use the *Time-series* resample option — be explicit which); classifications/sec at the deployed cadence; inference latency mean from `result.timing` (≥ 100 windows). Then **compute (do not measure)** an analytical energy estimate from datasheets: BMI270 active current at each ODR (from the BMI270 datasheet — *accel-only* is on the order of hundreds of μA ; *gyro adds* ~700–800 μA); nRF52840 active current at 64 MHz (~7–8 mA from the nRF52840 product spec, no radio). Compute energy per inference (mJ) = $(I_{\text{imu}} + I_{\text{mcu}}) \times 3.3 \text{ V} \times \text{inference_time}$, and energy per second of continuous operation using your measured throughput. Tabulate (accuracy, throughput, latency, computed mJ per inference) across the three rates. Two paragraphs: (a) **where is the knee of the curve?** Which sample rate maximizes accuracy per mJ — highest, lowest, or middle? Tie to the L10 operating-point framing (HW 4 Q5). (b) **one specific deployment scenario** where the *low-rate* point wins on a coin-cell budget (HW 4 Q14), citing the specific BMI270 or nRF52840 number that drives your choice. **Deliverable:** the measurement-and-computation table, cited datasheet pages, and the two analysis paragraphs.

Hints:

Skeleton sketches with `// TODO(student)` markers are under `lab-3/starter/`; build each unmodified first to confirm your toolchain, then fill the TODOs. **Edge Impulse Studio sorts class labels alphabetically** when generating `model_metadata.h`, so for the suggested 4-class set the order is `circle` (0), `flex` (1), `idle` (2), `punch` (3) — verify in your generated `ei_classifier_inferencing_categories[]` and set `kIdleClassIx` in your sketch to match (if your set is different, this index will be different). **Set the Spectral Analysis FFT length to ≥ 32** in the EI impulse design — the nRF52840's hardware FFT requires a power of 2 in [32, 4096]; a smaller FFT will silently fall back and your model will see degenerate features (a symptom is `top=idle conf=1.00` on every window regardless of motion). For Option C.1, if Chrome's picker shows

the device as “Arduino” instead of “Nano33Gesture,” that’s a 31-byte advertisement-packet overflow — the reference sketch sets the **GAP Device Name** via `BLE.setDeviceName()` to fix this; verify your sketch does the same.

Lab Materials:

Required Parts:

Part name	Quantity	Notes
Arduino Nano 33 BLE Sense Rev2 (SKU ABX00070)	1	with headers; on-board BMI270 IMU + RGB LED + BLE radio
USB-A to micro-USB cable	1	must be data -capable, not power-only

No breadboard or external components are required — the lab uses **on-board** sensors only. Option C.1 additionally needs a **laptop running Chrome or Edge** (or an Android phone with Chrome) for the Web Bluetooth receiver page; iPhone-only students can use **Bluefruit LE Connect** (open-source) as a fallback.

Required Software:

- (a) Arduino IDE 2.x (<https://www.arduino.cc/en/software>) with the **Arduino Mbed OS Nano Boards** package (from Lab 1).
- (b) `Arduino_BMI270_BMM150` library (on-board IMU + magnetometer) — install via the IDE Library Manager.
- (c) An **Edge Impulse** account (free) — <https://edgeimpulse.com>; reuse your Lab 2 Track K account if you have one.
- (d) The **Edge Impulse CLI** (for the data forwarder; `npm install -g edge-impulse-cli`), or the Edge Impulse phone app for sample collection.
- (e) The Edge Impulse-generated **Arduino library**, exported from your project and added via **Sketch** → **Include Library** → **Add .ZIP Library...**

Option C.1 (5862) only:

- (f) `ArduinoBLE` library — install via the IDE Library Manager.
- (g) A laptop running **Chrome or Edge** (or an Android phone with Chrome) for the Web Bluetooth receiver page in `tools/web-ble-receiver.html`.

Lab Instructions

Step 1:

Install the toolchain. Arduino IDE 2.x with the Mbed OS Nano Boards package is the same as Lab 1. Add the `Arduino_BMI270_BMM150` library via the Library Manager. Sign in to (or create) your Edge Impulse account, install the Edge Impulse CLI (or download the phone app). For Option C.1 (5862), also install the `ArduinoBLE` library.

Step 2:

Complete **Part A.1** (requirement (1)): flash `starter/g1-imu-capture/` from the lab repo to the Nano, open the Serial Monitor at **115200 baud** (not 9600), and confirm a CSV header followed by streaming `millis,ax,ay,az,gx,gy,gz` lines. Wave the board and confirm the accel values move between $\sim\pm 1$ g; rotate it and confirm the gyro values move. Measure the actual sample rate by counting lines over one second.

Step 3:

Complete **Part A.2** (requirement (2)): write the Spectral Analysis explanation and the raw → features diagram for your report.

Step 4:

Complete **Part B.1** (requirement (3)): in Edge Impulse Studio, create a project (device type *Arduino Nano 33 BLE Sense*); collect $\geq \sim 30$ examples per class for your four classes using the data forwarder against your A.1 sketch (or the phone app); design the impulse (Spectral Analysis + Classification); set the Spectral Analysis **FFT length ≥ 32** ; train ~ 100 epochs; record validation accuracy + confusion matrix; build a **Quantized (int8)** Arduino library.

Step 5:

Complete **Part B.2** (requirement (4)): install the EI library zip via *Sketch → Include Library → Add .ZIP Library...*; in `starter/g2-gesture-inference/`, change the `#include` to your EI project's `<your-project-name>_inferencing.h`; set `kIdleClassIx` to your idle class's index in `ei_classifier_inferencing_category` (alphabetical → typically 2 for a `circle/flex/idle/punch` set); complete the IMU sample loop, the top-class + confidence-gate logic, the test-and-trace filter, and the label-aware act stage; upload and verify gestures blink the matching color and `idle` does nothing.

Step 6:

Complete **Part B.3** (requirement (5)): record the **baseline** measurement table (accuracy, latency mean, classifications/sec, flash, RAM) and a separate row for the **filter-added** detection latency (`window_stride × kAgreeMin`); apply your chosen optimization (i, ii, or iii); re-deploy and re-measure; write the L9/L10 analysis paragraph. Print unsigned long timing with `Serial.print()` (not `MicroPrintf %lu`), and remember the nRF52840's `micros()` has $\sim 4 \mu\text{s}$ resolution, so the **mean** over many windows is the meaningful number.

EECE5862:

<For 5862 students>: Complete **one** of options **C.1** or **C.2** from requirements (6)–(7). Document your method, results, and analysis paragraphs in the lab report. For C.1, start from `starter/g3-ble-telemetry/` and use the provided Web Bluetooth receiver in `tools/web-ble-receiver.html`. For C.2, you only need the on-board measurements plus published datasheet currents — no power meter required.

Step 7:

Test and debug your design. Write your lab report following the **Lab Report Template** posted on Canvas. Include all required deliverables: the four classes you trained on the title page, the feature diagram, EI training/accuracy evidence (Model page + Feature explorer screenshots), the two measurement tables (baseline + after-optimization), serial/video evidence of live detection, code listings for the parts you wrote (screenshots of key snippets — do not paste full dumps), and your analysis paragraphs. Per the syllabus **AI Use Policy**, disclose any AI tool you used (tool, prompts, output, and modifications).

Step 8:

Push your source code to a **private** GitHub repository. Submit the lab report PDF and a .zip of your Arduino sketches via Canvas. Also include a **link to your public Edge Impulse project** (or the exported library zip). For Option C.1, include your screenshot of the Web Bluetooth receiver page showing the byte stream as you gestured.

Step 9:

Record a short demo video showing your working setup — each gesture triggering its matching LED color and (5862 C.1 only) the receiver page logging the corresponding byte — and post it on YouTube. Include the video URL in your lab report.

###

Please refer to lab demonstration and report guidelines for demonstration and report writing.